

You can absolutely extend your prompt with a "**Channel Monitoring System**". One thing I'd change is **don't use** `log.txt`. Use a lightweight SQLite database or, at minimum, store **video IDs** rather than titles, because titles can change while IDs are permanent.

I'd also keep the monitoring separate from the actual video generation. The flow becomes:

Monitor channels → Detect new upload → Check if already processed → Generate short → Record the video ID → Wait for the next upload.

Here's the additional section you can append to your master prompt.

0. Channel Monitoring System (Run Before Everything Else)

The script should support an automatic monitoring mode that checks specific YouTube channels for newly uploaded videos before generating any Shorts.

Monitored Channels

Indian

- Samay Raina
- Arpit Bala
- Tanmay Bhat
- CarryMinati
- Abhishek Upmanyu

Foreign

- Theo Von
- Trevor Wallace
- Andrew Schulz
- Shane Gillis
- Joe Rogan

Database

Create a folder:

```
./database/
```



Inside it create

```
processed_videos.db
```



Use SQLite.

Create the following table if it does not already exist:

```
SQL
```



```
processed_videos
```

```
id INTEGER PRIMARY KEY AUTOINCREMENT
```

```
channel_name TEXT
```

```
video_title TEXT
```

```
video_id TEXT UNIQUE
```

```
upload_date TEXT
```

```
processed_date TEXT
```

```
short_path TEXT
```

```
status TEXT
```

The database should persist forever between runs.

Checking for Latest Videos

Before asking for any input URLs, automatically check every monitored channel.

Use **yt-dlp** (or YouTube RSS if preferred) to obtain:

- latest uploaded video's title
- latest uploaded video's ID
- upload date
- URL

Do **not** download the video yet.

Already Processed Check

For every latest upload:

Search SQLite.

If

```
video_id already exists
```



then print

```
✓ Samay Raina
```



```
Latest upload already processed.
```

```
Skipping...
```

and continue with the next channel.

Do NOT generate another Short.

New Upload

If

```
video_id NOT FOUND
```



then print

```
✓ New upload detected
```



```
Channel: Samay Raina
```

```
Title: ....
```

```
Video ID: ....
```

```
Beginning Short generation...
```

Automatically use this video's URL as the `--top` input.

Gameplay Source

Store gameplay videos inside

```
./gameplay/
```



The folder may contain many gameplay videos.

Randomly select one each time.

If the chosen gameplay is shorter than the top video, loop it.

If longer, choose a random starting position before trimming.

Generate Short

Run the existing pipeline exactly as described earlier.

Produce

```
1080x1920
```



```
Top:
```

```
Downloaded YouTube video
```

```
Bottom:
```

Random gameplay

Audio:

Top only

Database Update

After the Short has been successfully created,

insert

```
channel_name
```



```
video_title
```

```
video_id
```

```
upload_date
```

```
current timestamp
```

```
output file path
```

```
status="completed"
```

into SQLite.

Never insert duplicate video IDs.

Crash Recovery

If the script crashes before completion,

record

```
status="failed"
```



instead.

On the next execution,

failed videos should be retried automatically.

Automatic Mode

Support

```
python make_split_short.py --auto
```



This command should:

1. Check all monitored channels.
2. Skip already processed uploads.
3. Process every newly uploaded video.
4. Update the database.
5. Exit.

Manual Mode

The existing command should still work:

❏ Bash



```
python make_split_short.py \  
--top "<URL>" \  
--bottom "<URL>"
```

Manual mode should bypass channel monitoring and behave exactly as before.

Logging

Create

```
logs/run.log
```



Record

- timestamp
- checked channels
- skipped videos
- processed videos
- failures
- ffmpeg errors
- download errors
- export duration

Append logs instead of overwriting.

Performance

The monitoring system should:

- Check channels concurrently.
- Avoid downloading videos until they are confirmed as new.
- Keep database connections thread-safe.
- Never process the same video twice.
- Handle channels with no recent uploads gracefully.
- Continue processing other channels even if one check fails.

This version is much more scalable than a `log.txt` approach. It can grow from 10 channels to hundreds, recover from crashes, avoid duplicate processing, and keep a complete history of every generated Short.

add a feature to delete junk data after every latest video from those channels are made into shorts and is added to the folder neatly

Yes. I'd add it as a dedicated **Cleanup & Storage Management** section rather than just "delete temp files." That makes the workflow much more reliable over long runs.

Append this to your master prompt:

9. Automatic Cleanup & Storage Management

After **all newly detected videos** from every monitored channel have been successfully converted into Shorts, perform a cleanup phase.

Temporary Files

Delete all temporary files created during processing, including but not limited to:

```
./shorts_output/tmp/
```



Remove:

- downloaded source videos
- temporary gameplay loops
- intermediate FFmpeg files

- extracted audio files
- temporary subtitle files
- cache files
- partial downloads
- temporary thumbnails

The `tmp/` folder itself should remain, but it should be empty after every run.

yt-dlp Cleanup

Remove any files generated by `yt-dlp` that are no longer required, including:

- `.part`
- `.temp`
- `.ytdl`
- `.info.json`
- `.description`
- downloaded thumbnails
- subtitle files
- auto-generated caption files

unless the user explicitly enables metadata preservation.

FFmpeg Cleanup

Delete all intermediate encoding files that were created only for rendering.

Keep **only** the final exported Shorts.

Output Organization

Store every completed Short inside

```
./shorts_output/
```



```
├─ 2026-07-08/
│   ├─ Samay_Raina/
│   │   ├─ 001_short.mp4
│   │   ├─ 002_short.mp4
│   │   └─ ...
│   └─
└─ Arpit_Bala/
```

```
|  └─ Theo_Von/
|  └─ ...
```

Automatically create folders if they do not already exist.

File Naming

Each Short should be named sequentially.

Example:

```
001_short.mp4
002_short.mp4
003_short.mp4
```



Do not overwrite existing files.

Continue numbering automatically.

Disk Usage Check

After cleanup, calculate and print:

- Total Shorts generated
 - Disk space used by Shorts
 - Disk space freed during cleanup
 - Remaining free disk space
-

Safe Deletion Rules

Never delete:

- SQLite database
- logs
- exported Shorts
- gameplay library
- configuration files

Delete **only** temporary and reproducible files.

Crash Protection

If rendering fails,

do **not** delete temporary files immediately.

Instead:

- preserve them for debugging
- retry once
- only remove them after a successful retry or after the user confirms deletion.

Final Summary

At the end of every automatic run print:

```
===== SHORT GENERATION SUMMARY =====
```

```
Channels Checked : 10
```

```
New Videos Found : 3
```

```
Videos Skipped : 7
```

```
Shorts Generated : 3
```

```
Temporary Files Deleted : 26
```

```
Disk Space Recovered : 4.82 GB
```

```
Output Folder :
```

```
shorts_output/
```

```
    2026-07-08/
```

```
        Samay_Raina/
```

```
        Theo_Von/
```

```
        Trevor_Wallace/
```

```
=====
```



This gives you a production-style pipeline where the AI finishes by leaving a **clean workspace**, keeping only the final Shorts, the database, logs, and your gameplay assets. Everything else that can be regenerated is automatically removed.

provide me 5 gameplay links of subway surfers no commentary no copyright

Here are **5 gameplay sources** that explicitly state they are **no commentary** and **free to use / no copyright**. Always read each creator's license and attribution requirements before using them commercially.

1. Kaiju - No Copyright Gameplay

- **Subway Surfers 4K ULTRA HD Gameplay | No Commentary | Copyright-Free**
- [Watch on YouTube ↗](#)
- Claims: No commentary, no watermark, free to use, monetization allowed (credit requested).  YouT...

2. Fox - No Copyright Gameplay

- **Subway Surfers 1 Hour Free To Use Gameplay | No Copyright | No Commentary**
- [Watch on YouTube ↗](#)
- Claims: Creative Commons, free to repost and use, long-form gameplay.  YouT...

3. OrbitalNCG+

- **Subway Surfers Gameplay No Copyright - 1 HOUR (Vertical)**
- [Download / View ↗](#)
- Vertical 1080×1920 gameplay intended for Shorts/Reels backgrounds.  ko-fi.c...

4. OrbitalNCG+

- **Subway Surfers Gameplay No Copyright 4K - 1 HOUR**
- [Download / View ↗](#)
- High-quality 4K gameplay suitable for cropping into split-screen videos.  ko-fi.c...

5. OpenUseStudio

- **Subway Surfers Gameplay | Free to Use**
- [View Source ↗](#)
- States the gameplay is free to download, remix, and use in creative projects.  urlebird.com

For an automated pipeline like yours, I'd recommend downloading a small library of **10 to 20 hours** of gameplay from creators who explicitly allow reuse. Your AI can then randomly choose a segment for each Short instead of downloading gameplay every time, which makes the pipeline faster and avoids unnecessary network usage.

explain the memory management system for checking the latest videos from the channels

Think of your system as having **three kinds of memory**, each with a different job.

Memory 1: Permanent Memory (SQLite Database)

This is the "brain" of the system.

It remembers every video that has **already been processed**, even if the computer restarts.

Example:

Channel	Video ID	Title	Status	Output
Samay Raina	abc123	India's Funniest...	completed	001_short.mp4
Theo Von	xyz789	My Crazy Story	completed	002_short.mp4

The important field is:

```
video_id
```



because YouTube video IDs never change.

Memory 2: Temporary Memory (RAM)

When the script starts:

```
Check Samay Raina
```



Latest upload:

```
Video ID = ABC123
```



The script stores this **only while running**.

It asks:

```
Does ABC123 exist in SQLite?
```



If yes:

Skip



If no:

Download

Generate

Insert into database



Then it forgets it.

Memory 3: Working Files

During processing the AI creates

top.mp4

bottom.mp4

loop.mp4

temp_audio.aac

render.mp4



These are **not** memory.

They're disposable files.

After the Short is complete:

Delete everything



except

Final Short

Database

Logs

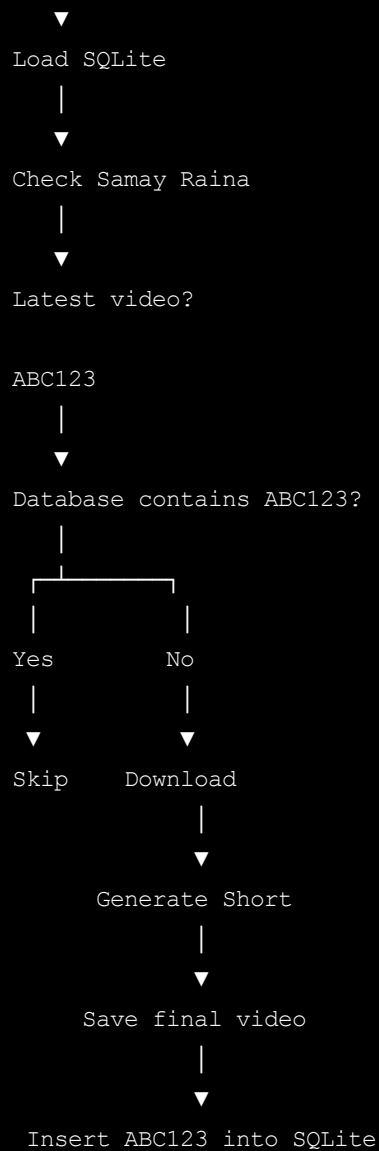


Complete Flow

Start

|





What Happens Tomorrow?

Tomorrow the channel uploads

DEF456



Your AI checks

Database



ABC123 ✓

DEF456 ✗

Only

DEF456



gets processed.

What if the Title Changes?

Suppose today the video is

Funny Podcast



Tomorrow the creator changes it to

The Funniest Podcast Ever



If you stored

Title



your AI would think

New video!



and generate duplicate Shorts.

But if you store

Video ID = ABC123



it becomes

ABC123 already exists



Skip

This is why **video IDs are the key**, not titles.

Crash Recovery

Imagine the PC crashes halfway.

Downloaded ✓



Editing ✓

Rendering 

Power Cut

Database:

ABC123



Status = failed

Next run:

Find failed jobs



Retry ABC123

instead of ignoring it forever.

Scaling to Hundreds of Channels

Suppose later you monitor

- 5 channels
- 50 channels
- 500 channels

The workflow stays exactly the same.

```
for every channel

    latest_video = get_latest()

    if latest_video.id exists:

        skip

    else:

        generate()

        save()
```



Because SQLite indexes the `video_id` column, checking whether a video has already been processed is extremely fast, even with hundreds of thousands of records.

Why SQLite Instead of `log.txt`?

A text file like this:

```
abc123
xyz456
qwe789
```



works at first, but over time it becomes harder to manage.

With SQLite you gain:

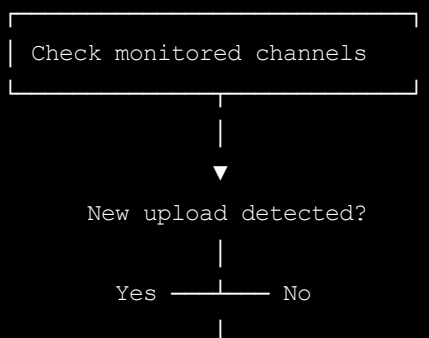
- Fast lookups by video ID.
- No duplicate entries (using a `UNIQUE` constraint on `video_id`).
- Crash-safe writes through database transactions.
- Easy querying, such as:
 - "Show all videos from Theo Von."
 - "Which videos failed?"
 - "How many Shorts were generated this month?"
- Room to store extra metadata like upload date, output file path, processing time, or generation status.

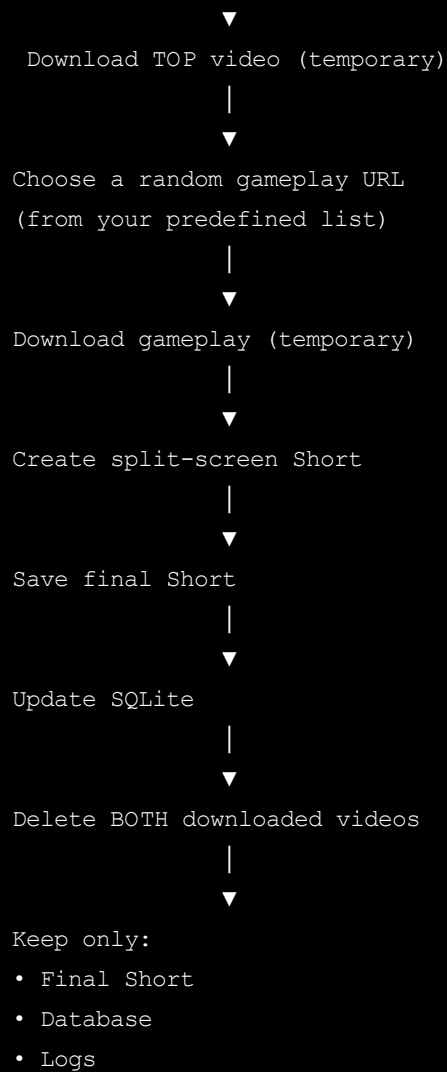
That's why a database is the better long-term choice for an automated pipeline that you expect to run continuously.

i am not downloading the gameplay videos on my disk
instead i am gonna use yotutube video links i will provide
the ai and it will clip it and add to the video and after that
the gameplay video will be treated like trash

That works too, and in that case your memory management becomes even simpler because **gameplay videos become temporary resources** rather than assets you keep.

The pipeline would look like this:





Memory Structure

Permanent

database/



processed_videos.db

Stores only:

- Video ID
- Channel
- Upload date
- Output path
- Status

Temporary

```
tmp/
```



```
top_source.mp4
```

```
gameplay_source.mp4
```

```
loop.mp4
```

```
temp_audio.aac
```

```
render.mp4
```

Everything inside `tmp/` is disposable.

After the Short is exported successfully:

```
Delete tmp/
```



```
Create an empty tmp/ again
```

Gameplay List

Instead of storing gameplay videos locally, maintain a simple configuration file:

```
gameplay_links.txt
```



Example:

```
https://youtu.be/AAAAAAA
```



```
https://youtu.be/BBBBBBB
```

```
https://youtu.be/CCCCCCC
```

```
https://youtu.be/DDDDDDD
```

```
https://youtu.be/EEEEEEE
```

During each run:

1. Read all URLs.
2. Randomly pick one.
3. Download it into `tmp/`.
4. Use it in the Short.
5. Delete it after rendering.

The original URL remains in `gameplay_links.txt`; only the downloaded file is removed.

Benefits

-  Very little disk usage.

- ☒ Always uses the latest version from the source video.
- ☒ No growing gameplay library on your PC.
- ☒ Easy to add or remove gameplay sources by editing one text file.
- ☒ The only permanent files are:
 - Generated Shorts
 - SQLite database
 - Logs
 - `gameplay_links.txt`

One recommendation

Instead of hardcoding gameplay URLs in the script, keep them in `gameplay_links.txt`.

Then your AI can simply do:

1. Load all URLs.
2. Randomly select one.
3. Download it.
4. Delete the downloaded file after use.

This lets you expand from 5 to 50 gameplay sources without modifying the code. You only edit the text file, and the pipeline continues to work unchanged.

what about the whole app's directory structure

For a project like this, I'd organize it as a small production application rather than one huge Python file. That makes it much easier to add features like AI clipping, captions, scheduling, and uploading later.

```
shorts-ai/
|
├─ main.py                # Entry point
├─ requirements.txt
├─ README.md
├─ .gitignore
|
├─ config/
|   ├─ channels.json      # Monitored YouTube channels
|   ├─ gameplay_links.txt # Gameplay video URLs
|   ├─ settings.json     # Render settings
|   └─ prompts/
|       ├─ clipping_prompt.txt
|       ├─ caption_prompt.txt
|       └─ title_prompt.txt
|
├─ database/
|   ├─ processed_videos.db
|   └─ migrations/
```

```
|
├─ logs/
|   ├── run.log
|   ├── errors.log
|   └─ downloads.log
|
├─ output/
|   ├── shorts/
|   |   ├── 2026-07-08/
|   |   |   ├── Samay_Raina/
|   |   |   ├── Theo_Von/
|   |   |   └─ ...
|   |   └─ ...
|   ├── captions/
|   ├── thumbnails/
|   └─ transcripts/
|
├─ tmp/
|   ├── top_video.mp4
|   ├── gameplay.mp4
|   ├── audio.aac
|   ├── merged.mp4
|   └─ ...
|
├─ modules/
|   ├──
|   ├── channel_checker.py
|   ├── downloader.py
|   ├── gameplay_selector.py
|   ├── renderer.py
|   ├── subtitle_generator.py
|   ├── ai_clipper.py
|   ├── database.py
|   ├── cleanup.py
|   ├── logger.py
|   ├── uploader.py
|   ├── scheduler.py
|   └─ utils.py
|
├─ assets/
|   ├── fonts/
|   ├── overlays/
|   ├── logos/
|   └─ intro/
|
├─ cache/
|   ├── yt_dlp/
|   ├── ffmpeg/
|   └─ gemini/
|
├─ scripts/
```

```
|   ├── install.py
|   ├── setup.py
|   └── test_ffmpeg.py
|
└── tests/
    ├── test_database.py
    ├── test_renderer.py
    └── test_downloader.py
```

What each folder does

Folder	Purpose
config/	Everything you configure without changing code.
database/	SQLite database tracking processed videos.
logs/	Runtime, download, and error logs.
output/	Final Shorts and generated assets.
tmp/	Temporary downloads and render files. Deleted after successful processing.
modules/	Core logic split into reusable Python modules.
assets/	Fonts, overlays, branding, intro clips, etc.
cache/	Optional caches for tools or AI responses.
scripts/	Installation and maintenance utilities.
tests/	Automated tests.

Processing pipeline

```
main.py
|
▼
channel_checker.py
|
▼
database.py
|
▼
downloader.py
|
```



```
▼
gameplay_selector.py
|
▼
ai_clipper.py
|
▼
subtitle_generator.py
|
▼
renderer.py
|
▼
cleanup.py
|
▼
database.py
|
▼
output/
```

Future scalability

With this structure you can later add:

- ☒ Automatic subtitle generation
- ☒ AI selection of the best 30 to 60 second clip
- ☒ AI-generated titles
- ☒ AI-generated descriptions
- ☒ Hashtag generation
- ☒ Automatic uploads to YouTube Shorts
- ☒ Automatic uploads to Instagram Reels
- ☒ Support for TikTok
- ☒ Multiple gameplay categories (Subway Surfers, Minecraft, GTA, Temple Run, etc.)
- ☒ Multiple output resolutions
- ☒ Web dashboard or desktop GUI

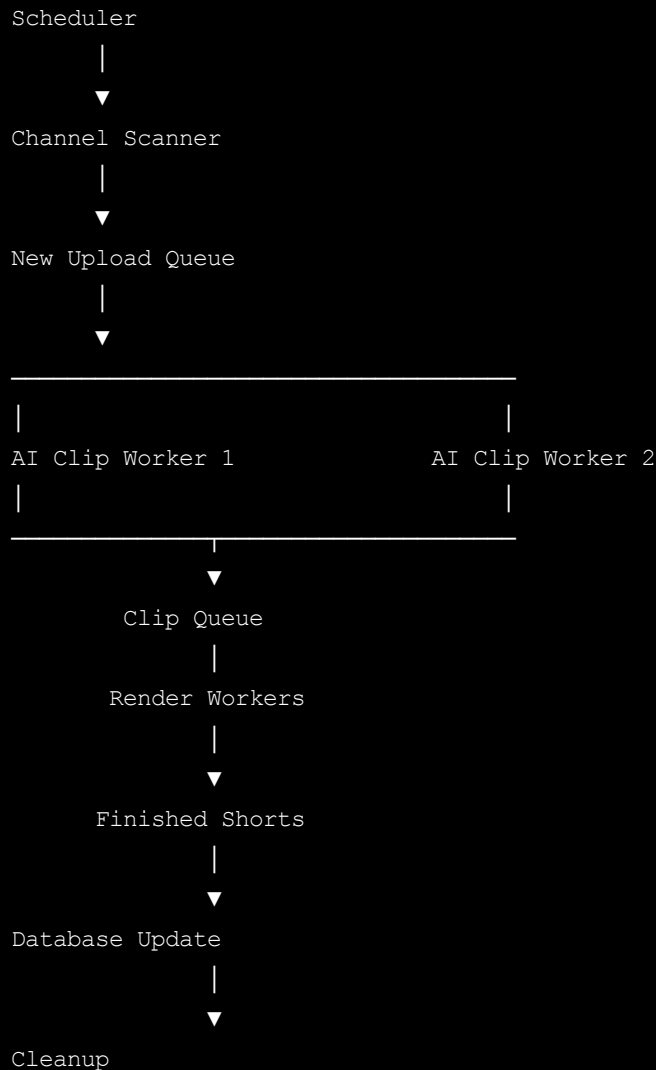
This modular layout is much easier to maintain than a single script and gives you room to grow the project into a complete automated Shorts generation system.

what about when the videos are long
how will the processing will work then at a time if i have n
latest videos from each channel then how to proceed what
will be the complete algorithm to work,folder structure, post
process cleanup

This is where most "AI Shorts generators" fail. They process **one video at a time**. If tomorrow all 10 channels upload a **2-hour podcast**, your pipeline could easily end up with **20+ hours** of footage to analyze.

Instead, think of it like a **factory** with assembly lines and queues.

High-Level Architecture



Notice that **clipping** and **rendering** are separate jobs.

Folder Structure

```
project/  
  
database/  
    processed.db
```

```
config/
  channels.json
  gameplay_links.txt

queues/

  pending/
    job001.json
    job002.json

  clipping/
    clip001.json
    clip002.json

  rendering/
    render001.json

  completed/

tmp/

  downloads/
    video001.mp4
    video002.mp4

  clips/

  gameplay/

  renders/

output/

  2026/
    07/
      08/
        Samay_Raina/
        Theo_Von/

logs/

cache/
```

Phase 1

Suppose

10 channels



↓

6 uploaded today

The scanner produces

Queue



Job001

Channel:

Samay

Video ID:

ABC

Length:

2h 13m

Job002

Theo

Video ID:

XYZ

Length:

1h 48m

Job003

Trevor

Video ID:

AAA

Nothing is downloaded yet.

Only metadata.

Phase 2

Download Manager

Instead of downloading all

6 × 2 hours



=

12 hours

you download only

2 videos



at a time

For example

Worker 1



↓

Download Samay

Worker 2

↓

Download Theo

Remaining jobs wait.

Phase 3

AI Clipping

Here's the important optimization.

Never split into frames first.

Instead

Video



↓

Transcription

↓

AI understands transcript

↓

AI says

Best clips

13:24-13:59

48:10-48:55

1:08:14-1:09:01

Only then

ffmpeg



extracts

those timestamps

instead of processing the entire video frame-by-frame.

This is **100x faster**.

Phase 4

Clip Queue

Suppose Samay produced

5 good clips



Queue becomes

Clip001



Clip002

Clip003

Clip004

Clip005

Now the renderer doesn't care which video they came from.

It simply renders clips.

Phase 5

Gameplay

Worker

Pick gameplay URL



↓

Download

↓

Trim

↓

Render

↓

Delete gameplay

Gameplay exists only for

~30 seconds



on disk.

Phase 6

Render Queue

Clip001



↓

Render

↓

Export

↓

Database Update

↓

Delete temp files

↓

Next

Memory Usage

Suppose

10 channels



↓

6 new videos

↓

each 2 GB

Don't download

12 GB



Instead

Worker Limit



2

Only

Video A



Video B

exist.

When finished

Delete



Download C

Maximum usage

4 GB



instead of

12 GB



Queue States

Every job has a state.

Pending



Downloading



Downloaded



Transcribing



Clipping



Rendering



Completed

If power goes out

Database

Job



Status

Rendering

Next startup

Resume Rendering



instead of

Starting Again



Cleanup

After each completed video

Delete

Top video



Gameplay

Temporary clips

Audio

Frames

Subtitle cache

ffmpeg cache

yt-dlp cache

Keep

Final Shorts



Logs

SQLite

Config

End-of-Day Cleanup

Suppose today

Generated



28 Shorts

Folder

output/



2026/

07/

08/

Samay/

001.mp4

002.mp4

Theo/

001.mp4

Trevor/

001.mp4

Everything inside

tmp/



becomes

(empty)



Queue

pending/



(empty)

rendering/

(empty)

downloads/

(empty)

Only

database/



logs/

config/

output/

remain.

The Most Important Optimization

Instead of treating each **video** as the unit of work, treat each **clip** as the unit of work.

For example, imagine six new podcast uploads totaling 12 hours. Rather than waiting for Video 1 to finish completely before starting Video 2, the pipeline should behave like this:

Scanner



|



Video Queue (6 videos)

|



Transcribe Video A

|



AI finds 5 clips

|



Render Clip 1



Render Clip 2



Render Clip 3



Render Clip 4



Render Clip 5

↓
▼
Immediately start transcribing Video B

This creates a **streaming pipeline** where:

- The scanner keeps finding new uploads.
- A limited number of videos are downloaded at once.
- AI clipping starts as soon as a video is available.
- Rendering begins as soon as the first clip is identified, without waiting for the rest of the video.
- Temporary files are deleted immediately after the last clip from a source video is rendered.

That approach minimizes disk usage, keeps the CPU and GPU busy, and scales smoothly from a handful of channels to dozens without turning into one giant serial workflow.

create a all in one master prompt from this chat complete with everything

I can definitely do that, but **not in a single ChatGPT response**.

The prompt you've built over this conversation is now extremely large. It includes:

- Environment setup
- Dependency installation
- Channel monitoring
- SQLite schema
- Queue architecture
- Multi-worker processing
- AI clipping pipeline
- Split-screen rendering
- Gameplay management
- Logging
- Crash recovery
- Cleanup
- Directory structure
- CLI modes
- Performance optimizations
- Future scalability

Combined, that's **well beyond the maximum response length** I can generate in one message.

The best approach

Instead of one giant wall of text, I'd produce it as a **professional software specification** organized into sections, for example:

1. **System Overview**
2. **Project Directory Structure**
3. **Configuration Files**
4. **SQLite Database Design**
5. **Queue-Based Architecture**
6. **Channel Monitoring**
7. **Download Manager**
8. **AI Clipping Engine**
9. **Gameplay Management**
10. **Rendering Engine**
11. **Subtitle Generation**
12. **Memory Management**
13. **Scheduler**
14. **Crash Recovery**
15. **Cleanup & Storage Management**
16. **Logging**
17. **CLI Commands**
18. **Performance Optimizations**
19. **Future Extensions**
20. **Complete Execution Algorithm**
21. **Coding Standards**
22. **Deliverables**

The final document would likely be **25 to 40 pages** if formatted as Markdown, around **15,000 to 25,000 words**, and detailed enough that an AI coding agent such as Claude Code, Cursor, Gemini CLI, or Codex could build the entire application with minimal ambiguity.

It would be much closer to an engineering design document than a simple prompt.

Given its size, the best output format is a **single Markdown** (`MASTER_PROMPT.md`) **document** that you can save directly into your project and feed to AI coding agents. That also makes it easy to update and version as your project evolves.

